

App Corretora — Funcionamento da Aplicação

Como a aplicação está estruturada hoje, com foco em **login** e **cadastro**: os três perfis de usuário, os fluxos de autenticação e registro, e as regras que o banco de dados garante.

Stack: React + Vite + TypeScript (SPA) · Express 5 + `tsx` (API REST) · PostgreSQL/Neon via Prisma Next (`@prisma/orm-postgres`)

Gerado em 03/09/2026 · Branch `master`

Sumário

1. Visão geral
2. Perfis de usuário
3. Autenticação (login)
4. Cadastro — os três caminhos
5. Regras garantidas pelo banco
6. Modelo de dados e posse do imóvel
7. Referência da API
8. Como rodar e contas de teste

1. Visão geral

A aplicação atende **imobiliárias** e **corretores**. Uma imobiliária se cadastra, vira administradora da própria conta e cadastra seus corretores; cada corretor registra seus imóveis, e a imobiliária também registra imóveis (próprios ou atribuídos a um corretor). Existe ainda o **corretor autônomo**, sem vínculo com nenhuma imobiliária.

Front-end — `client/`

- SPA em React + Vite + TypeScript, com `react-router`.
- Em dev: Vite na porta `5173` faz proxy de `/api` para a API na `3000`.
- Em produção: `npm run build` gera `client/dist`, servido pelo próprio Express com fallback de SPA.
- Estado de sessão no contexto `AuthProvider`; rotas protegidas por `RequireAuth` / `RequireAdmin`.

Back-end — `src/`

- Express 5 rodando com `tsx`. API REST sob `/api/*`.
- Sessão = cookie `httpOnly` assinado com HMAC (`src/auth.ts`) — sem biblioteca de auth.
- Senha com `scrypt` nativo do Node.
- Acesso ao banco via `db.orm.public.<Modelo>` (Prisma Next).

2. Perfis de usuário

Todo mundo que faz login é uma linha na tabela `usuario`, diferenciada pela coluna `tipo_usuario` (enum `TipoUsuario`). A imobiliária *não* tem login próprio: ela é uma linha em `empresa` (dados/CNPJ) + uma linha `usuario` do tipo `ADMIN` ligada a ela.

Perfil	Como nasce	Vínculo com empresa	O que faz
<code>ADMIN</code> a imobiliária	Cadastro público de imobiliária (cria <code>empresa</code> + este usuário)	Obrigatório. No máximo 1 ADMIN por imobiliária.	Cadastra e edita corretores; cadastra imóveis da imobiliária ou atribuídos a um corretor; vê todos os imóveis da empresa.
<code>CORRETOR</code> corretor da imobiliária	Criado pelo <code>ADMIN</code> já logado (não há auto-cadastro)	Obrigatório — herda a empresa do ADMIN que o criou.	Cadastra, edita e exclui os <i>seus</i> imóveis. Vê só os seus.
<code>CORRETOR_INDIVIDUAL</code> corretor autônomo	Cadastro público (<code>/api/registrar</code>)	Nunca. <code>id_empresa</code> fica <code>NULL</code> .	Cadastra, edita e exclui os seus imóveis. Vê só os seus.

3. Autenticação (login)

3.1 Como a senha é guardada

Ao cadastrar, a senha passa por `hashPassword()` e é gravada no formato `scrypt$N$salt_hex$hash_hex` (parâmetro de custo `N = 16384`, chave de 32 bytes). O texto puro nunca é armazenado. Na verificação, `verifyPassword()` reprocessa a senha informada com o mesmo *salt* e compara em tempo constante (`timingSafeEqual`).

3.2 Como a sessão funciona

Não há sessão em servidor nem JWT de biblioteca. `createSession()` monta um token no estilo "JWT enxuto": `base64url(JSON) . HMAC-SHA256(SECRET, corpo)`. O corpo carrega `sub` (id do usuário), `nome`, `email`, `tipoUsuario`, `empresaId` e `exp`. Esse token vai num cookie:

Nome `corretora_session`

Flags `HttpOnly`, `Path=/`, `SameSite=Lax`, `Max-Age` = 8 horas; `Secure` quando `NODE_ENV=production`

Validação a cada requisição, `loadSession` confere a assinatura HMAC e a expiração; se falhar, `req.session` fica indefinido

Autorização. `requireAuthApi` exige sessão válida (senão `401`). `requireAdminApi` exige `tipoUsuario === "ADMIN"` com `empresaId` definido (senão `403`). O `tipoUsuario` e o `empresaId` ficam *dentro* do token assinado — o cliente não consegue forjá-los.

3.3 Fluxo do login — `POST /api/login`

1	Usuário tela /login	Preenche e-mail e senha e envia o formulário.
2	Front (SPA)	<code>POST /api/login</code> com <code>{ email, senha }</code> em JSON.
3	API	Busca o usuário por e-mail. Se não existir, ainda roda um hash "descartável" para nivelar o tempo de resposta (evita enumeração por <i>timing</i>).
4	API	<code>verifyPassword()</code> . E-mail ou senha inválidos → <code>401 { error }</code> .
5	API	OK: <code>startSession()</code> grava o cookie <code>corretora_session</code> com <code>tipoUsuario</code> e <code>empresaId</code> . Responde <code>200 { ok: true }</code> .
6	Front	<code>refresh()</code> chama <code>GET /api/me</code> , guarda o usuário no contexto e navega para <code>/app</code> .

3.4 Sessão no front

- `AuthProvider` chama `GET /api/me` ao montar; enquanto isso, `loading = true`.
- `RequireAuth`: sem usuário → redireciona para `/login`.
- `RequireAdmin`: sem usuário → `/login`; usuário não-ADMIN → `/app`. Protege a rota `/corretores`.
- `logout()` → `POST /api/logout` (limpa o cookie) e zera o contexto.

4. Cadastro — os três caminhos

A decisão de projeto foi **endpoints separados**. A tela `/registro` tem um seletor “Corretor autônomo” / “Imobiliária” que troca os campos e o endpoint. O cadastro de *corretor da imobiliária* não é auto-serviço: acontece dentro do painel, com o ADMIN logado.

Caminho	Endpoint	Quem chama	Cria	Abre sessão?
Corretor autônomo	POST <code>/api/registrar</code>	Visitante (público)	1 <code>usuario</code> <code>CORRETOR_INDIVIDUAL</code> , <code>id_empresa = NULL</code>	Sim — já entra logado
Imobiliária + admin	POST <code>/api/registrar-imobiliaria</code>	Visitante (público)	1 <code>empresa</code> + 1 <code>usuario</code> ADMIN ligado a ela (transação)	Sim — entra logado como ADMIN
Corretor da imobiliária	POST <code>/api/corretores</code> ADMIN	ADMIN logado (painel)	1 <code>usuario</code> <code>CORRETOR</code> na empresa do ADMIN	Não — o ADMIN continua na própria sessão

4.1 Corretor autônomo — `/api/registrar`

- Visitante**
`/registro` · aba “Corretor autônomo”
Informa nome, e-mail e senha (mín. 6 caracteres).
- Front**
`POST /api/registrar` com `{ nome, email, senha }`.
- API**
Valida os campos. E-mail já usado → `409`.
- API**
Cria `usuario` com `tipoUsuario: "CORRETOR_INDIVIDUAL"` e `empresaId: null`; senha via `hashPassword()`.
- API**
`startSession()` → cookie. Responde `201 { ok: true }`.
- Front**
`refresh()` + navega para `/app`.

4.2 Imobiliária + ADMIN — `/api/registrar-imobiliaria`

- Visitante**
`/registro` · aba “Imobiliária”
Informa nome e CNPJ da imobiliária, telefone (opcional) e os dados do administrador (nome, e-mail, senha).
- Front**
`POST /api/registrar-imobiliaria` com `{ nome, email, senha, empresaNome, cnpj, empresaTelefone }`.
- API**
Valida. E-mail já usado → `409`. CNPJ já cadastrado → `409`.
- API**
Transação: cria `empresa` (nome, CNPJ, telefone, e-mail) e, em seguida, `usuario` `ADMIN` com `empresaId` apontando para ela. Se qualquer passo falhar, nada é gravado.

5 API `startSession()` como ADMIN. Responde `201 { ok: true }`.

6 Front Entra no painel; a seção “Corretores” e o menu do ADMIN aparecem.

4.3 Corretor da imobiliária — `/api/corretores` ADMIN

1 ADMIN painel /corretores Preenche nome, e-mail, senha e, opcionalmente, telefone e CPF do novo corretor.

2 Front `POST /api/corretores` (cookie de sessão do ADMIN vai junto).

3 API `requireAdminApi`: sessão precisa ser ADMIN com empresa — senão `403`.

4 API Valida; e-mail já usado → `409`. Cria `usuario` `CORRETOR` com `empresaId` = empresa do ADMIN.

5 API Responde `201 { ok: true, id }`. **Não** mexe na sessão do ADMIN.

6 Corretor Passa a entrar normalmente pelo `/api/login` com o e-mail e a senha definidos pelo ADMIN.

Edição e remoção de corretor (também só ADMIN, e só sobre corretores da própria empresa): `PATCH /api/corretores/:id` altera nome, telefone e CPF (e-mail e senha não mudam por aqui). `DELETE /api/corretores/:id` remove o corretor numa transação e, antes de apagar, devolve os imóveis dele para o *pool* da imobiliária (`id_usuario` vira `NULL`, a empresa é mantida) e zera o `criado_por` que apontava para ele.

5. Regras garantidas pelo banco

Além da validação na API, o PostgreSQL rejeita estados inconsistentes:

Objeto	Regra	Por quê
<code>CHECK</code> <code>usuario_empresa_por_tipo</code>	<code>(tipo = 'CORRETOR_INDIVIDUAL' AND id_empresa IS NULL)</code> <code>OR (tipo IN ('ADMIN', 'CORRETOR') AND id_empresa IS NOT NULL)</code>	Autônomo nunca tem empresa; ADMIN e CORRETOR sempre têm.
<code>índice único parcial</code> <code>usuario_um_admin_por_empresa</code>	<code>UNIQUE (id_empresa) WHERE tipo_usuario = 'ADMIN'</code>	No máximo um ADMIN por imobiliária.
<code>CHECK</code> <code>usuario_tipo_usuario_check</code>	<code>tipo_usuario IN ('ADMIN', 'CORRETOR', 'CORRETOR_INDIVIDUAL')</code>	Enum <code>TipoUsuario</code> — só valores válidos.
<code>CHECK</code> <code>imovel_tem_posse</code>	<code>id_empresa IS NOT NULL OR id_usuario IS NOT NULL</code>	Todo imóvel pertence a uma imobiliária, a um corretor, ou aos dois.
<code>UNIQUE</code> em <code>usuario</code>	<code>email</code> , <code>cpf</code> , <code>cnpj</code> (os dois últimos aceitam múltiplos <code>NULL</code>)	Sem contas duplicadas.

6. Modelo de dados e posse do imóvel

empresa

`id_empresa` · `nome` · `cnpj` (único) · `telefone?` · `email?`

Uma imobiliária. Tem 0..N `usuario` e 0..N `imovel`.

usuario

`id_usuario` · `nome` · `email` (único) · `senha` (scrypt) · `tipo_usuario` · `id_empresa?` · `cpf?` · `cnpj?` · `telefone?` · `data_nasc?` · `instagram?`

Quem faz login. `id_empresa` nulo só para `CORRETOR_INDIVIDUAL`.

imovel — como a posse é representada

Situação	<code>id_usuario</code>	<code>id_empresa</code>
Imóvel de um corretor da imobiliária	o corretor	a imobiliária
Imóvel da imobiliária (pool, sem corretor)	<code>NULL</code>	a imobiliária
Imóvel de corretor autônomo	o corretor	<code>NULL</code>

`criado_por` registra quem cadastrou (relevante quando o ADMIN cadastra em nome de um corretor). `GET /api/imoveis` filtra por papel: ADMIN vê tudo da empresa; corretor (vinculado ou autônomo) vê só o que é seu.

7. Referência da API

Método / rota	Acesso	O que faz
POST /api/login	público	Autentica e abre a sessão (cookie).
POST /api/logout	público	Limpa o cookie de sessão.
GET /api/me	autenticado	Dados da sessão: <code>id</code> , <code>nome</code> , <code>email</code> , <code>tipoUsuario</code> , <code>empresaId</code> .
POST /api/registrar	público	Cria corretor autônomo e já loga.
POST /api/registrar-imobiliaria	público	Cria imobiliária + ADMIN (transação) e já loga.
GET /api/corretores	ADMIN	Lista os corretores da imobiliária.
POST /api/corretores	ADMIN	Cadastra um corretor na empresa do ADMIN.
PATCH /api/corretores/:id	ADMIN	Edita nome, telefone e CPF do corretor.
DELETE /api/corretores/:id	ADMIN	Remove o corretor; imóveis dele voltam ao pool.
GET /api/imoveis	autenticado	Lista imóveis conforme o papel (ADMIN = empresa; corretor = os seus).
POST /api/imoveis	autenticado	Cadastra imóvel; posse definida pelo papel de quem cadastra.
PUT /api/imoveis/:id	dono ou ADMIN da empresa	Edita o imóvel; só o ADMIN reatribui a corretor.
DELETE /api/imoveis/:id	dono ou ADMIN da empresa	Exclui o imóvel e suas características.
GET /api/bairros · /api/caracteristicas	autenticado	Listas de apoio para o formulário de imóvel.

Erros seguem o padrão `{ ok: false, error: "mensagem" }` com status `400/401/403/404/409/500`. Sucesso: `{ ok: true, ... }`.

8. Como rodar e contas de teste

- Subir tudo (dev) `npm run dev` — Vite em `:5173` (proxy `/api` → `:3000`) + API com `tsx watch`.
- Popular o banco `npm run seed` — idempotente.
- Build de produção `npm run build` → `client/dist`, servido por `npm start`.
- Migrações `npx prisma contract emit` → `npx prisma db update` (fluxo rápido) ou `migration plan` + `db migrate`.

Contas criadas pelo seed (senha `senha123`):

E-mail	Perfil	Empresa
<code>admin@demo.com.br</code>	<code>ADMIN</code>	Imobiliária Demo
<code>corretor@demo.com.br</code>	<code>CORRETOR</code>	Imobiliária Demo
<code>autonomo@demo.com.br</code>	<code>CORRETOR_INDIVIDUAL</code>	—